Portafolio de evidencias: Análisis inferencial de datos

Dataset: *Students Performance in Exams*

Curso: Fundamentos para IA **Actividad:** Semana 6 — Portafolio de evidencias (análisis inferencial de datos)

Este *notebook* contiene el procedimiento completo, reproducible, del análisis estadístico descriptivo e inferencial desarrollado sobre el *dataset* `StudentsPerformance.csv` (fuente: Kaggle - Students Performance in Exams).

Índice

0. Conceptos básicos de Estadística Inferencial y Test de Hipótesis
1. Carga del *dataset* y exploración inicial
2. Contexto y planteamiento del problema
3. Identificación y clasificación de variables
4. Descripción de la muestra y tamaño de los grupos
5. Estadística descriptiva y visualización
6. Validación de supuestos (normalidad y homogeneidad de varianzas)
7. Selección y justificación del análisis inferencial
8. Resultados e interpretación
9. Conclusiones

> 📘 **Versión explicada sencilla:** cada sección de este *notebook* tiene, justo después del análisis técnico, un recuadro **"👴 Explicado simple"** con la misma idea contada en palabras cotidianas. La versión completa, paso a paso, está en `Desarrollo_Actividad_Explicado_Nino.md` (misma carpeta).

0. Conceptos básicos de Estadística Inferencial y Test de Hipótesis

(El desarrollo conceptual completo se encuentra en el portafolio. Aquí se resume lo esencial para seguir el análisis.)

- **Estadística inferencial**: conjunto de técnicas que permiten sacar conclusiones sobre una **población** a partir de los datos observados en una **muestra**.
- **Hipótesis nula (H0)**: afirmación de "no efecto" o "no diferencia" que se asume verdadera hasta que la evidencia indique lo contrario.
- **Hipótesis alternativa (H1)**: afirmación que contradice a H0; es lo que se busca sustentar con evidencia.
- **Nivel de significancia (α)**: umbral de riesgo aceptado para rechazar H0 por error. En este trabajo se usa el estándar **α = 0.05**.
- **Estadístico de prueba y *p-value***: el estadístico resume la evidencia muestral; el *p-value* es la probabilidad de observar un resultado igual o más extremo si H0 fuera cierta.
  - Si *p-value* < α: se **rechaza H0** (evidencia estadísticamente significativa).
  - Si *p-value* ≥ α: **no se rechaza H0** (no hay evidencia suficiente).
- **Errores tipo I y II**: rechazar H0 siendo verdadera (falso positivo) o no rechazarla siendo falsa (falso negativo).
- **Supuestos de las pruebas paramétricas**: normalidad de la variable y homogeneidad de varianzas entre grupos. Si no se cumplen, se recurre a alternativas **no paramétricas** (p. ej. Mann-Whitney U en vez de la prueba t).

1. Carga del *dataset* y exploración inicial

In [1]:
```
# Estas líneas "traen las herramientas" que vamos a usar, como sacar las tijeras, la regla y los colores
# de la cartuchera antes de empezar un trabajo. Cada una sirve para algo distinto:
import pandas as pd              # pandas: para leer y ordenar la tabla de datos (como una hoja de Excel)
import numpy as np               # numpy: para hacer cuentas numéricas
from scipy import stats          # scipy.stats: trae las "pruebas estadísticas" que usaremos más adelante
import matplotlib.pyplot as plt  # matplotlib: para hacer dibujos y gráficas
import seaborn as sns            # seaborn: hace esas mismas gráficas pero más bonitas
from io import StringIO          # io.StringIO: nos ayuda a "limpiar" el archivo antes de leerlo (ver abajo)

sns.set_theme(style="whitegrid")       # le decimos que dibuje con un fondo de cuadrícula clarita
pd.set_option("display.max_columns", None)  # para que nos muestre TODAS las columnas de la tabla, sin recortar

# A veces el archivo CSV viene con una fila "basura" de más al principio (por ejemplo, si alguien lo abrió y
# lo volvió a guardar desde Excel), y puede usar coma "," o punto y coma ";" para separar los datos. Para que
# esto no nos rompa la lectura, primero buscamos en qué línea empieza de verdad la tabla (la que dice
# "gender" al inicio) y se la damos a pandas ya "limpia", dejando que detecte solo el separador correcto.
with open("StudentsPerformance.csv", encoding="utf-8") as f:
    lineas = f.readlines()
```

```
texto_limpio = "".join(lineas)

# Aquí abrimos la "caja" (el archivo StudentsPerformance.csv, ya limpio) y la guardamos con el nombre df
# (de "data frame", que es como pandas le dice a una tabla de datos).
df = pd.read_csv(StringIO(texto_limpio), sep=None, engine="python")

# .head() nos muestra solo las primeras 5 fichas de la caja, para echar un vistazo rápido sin ver las 1000.
df.head()
```

Out[1]:

	gender	race/ethnicity	parental level of education	lunch	test preparation course	math score	reading score	writing score
0	female	group B	bachelor's degree	standard	none	72	72	74
1	female	group C	some college	standard	completed	69	90	88
2	female	group B	master's degree	standard	none	90	95	93
3	male	group A	associate's degree	free/reduced	none	47	57	44
4	male	group C	some college	standard	none	76	78	75

In [2]:

```
# df.shape nos dice el "tamaño" de la tabla: cuántas filas (estudiantes) y cuántas columnas (datos de cada uno).
print(f"Dimensiones del dataset: {df.shape[0]} filas x {df.shape[1]} columnas")

# df.info() es como pedirle a la computadora una "ficha técnica" de la tabla: nombre de cada columna,
# cuántos datos tiene (para ver si falta alguno) y de qué tipo son (texto o número).
df.info()
```

```
Dimensiones del dataset: 1000 filas x 8 columnas
<class 'pandas.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 8 columns):
 #   Column                       Non-Null Count  Dtype
---  ------                       --------------  -----
 0   gender                       1000 non-null   str
 1   race/ethnicity               1000 non-null   str
 2   parental level of education  1000 non-null   str
 3   lunch                        1000 non-null   str
 4   test preparation course      1000 non-null   str
 5   math score                   1000 non-null   int64
 6   reading score                1000 non-null   int64
 7   writing score                1000 non-null   int64
dtypes: int64(3), str(5)
memory usage: 62.6 KB
```

In [3]:

```python
# df.isnull().sum() cuenta, columna por columna, cuántas fichas están "en blanco" (sin dato).
print("Valores nulos por columna:")
print(df.isnull().sum())

# df.duplicated().sum() cuenta cuántas fichas están repetidas (copiadas dos veces por error).
print(f"\nFilas duplicadas: {df.duplicated().sum()}")

# df.describe() saca, para cada columna de números, el promedio, cuánto varían, el mínimo y el máximo.
# Es como pedirle a alguien que resuma en una sola tabla cómo se ven en general las notas de todos.
df.describe()
```

```
Valores nulos por columna:
gender                         0
race/ethnicity                 0
parental level of education    0
lunch                          0
test preparation course        0
math score                     0
reading score                  0
writing score                  0
dtype: int64

Filas duplicadas: 0
```

Out[3]:

	math score	reading score	writing score
count	1000.00000	1000.000000	1000.000000
mean	66.08900	69.169000	68.054000
std	15.16308	14.600192	15.195657
min	0.00000	17.000000	10.000000
25%	57.00000	59.000000	57.750000
50%	66.00000	70.000000	69.000000
75%	77.00000	79.000000	79.000000
max	100.00000	100.000000	100.000000

Comprensión del *dataset*: contiene 1000 registros de estudiantes con variables demográficas y socioeducativas (género, etnicidad, nivel educativo de los padres, tipo de almuerzo, curso de preparación para el examen) y tres puntajes numéricos (matemáticas, lectura y escritura). No se observan valores nulos ni filas duplicadas, por lo que no fue necesario aplicar criterios de exclusión de datos.

> 👴 **Explicado simple:** imagina una caja con 1000 fichas, una por cada estudiante. Cada ficha dice si es niño o niña, cuánto estudiaron sus papás, si almorzó bien, si hizo un curso antes del examen, y sus notas. Contamos las fichas: son 1000 completas, ninguna en blanco. ¡Podemos trabajar con todas!

2. Contexto y planteamiento del problema

¿Qué representa el *dataset*? Registra el desempeño de 1000 estudiantes en tres exámenes estandarizados (matemáticas, lectura y escritura), junto con variables de contexto socioeducativo, entre ellas si el estudiante completó un **curso de preparación** para el examen.

Problema a responder con inferencia: se quiere establecer si la participación en el curso de preparación está asociada con un mejor desempeño en matemáticas, o si la diferencia observada entre quienes lo tomaron y quienes no podría deberse simplemente al azar muestral.

Pregunta investigable: ¿Existe una diferencia estadísticamente significativa en el puntaje de matemáticas (*math score*) entre los estudiantes que completaron el curso de preparación para el examen y los que no lo completaron?

Hipótesis:

- **H0:** la media del puntaje de matemáticas es igual entre el grupo que completó el curso de preparación y el grupo que no lo completó (μ_completed = μ_none).
- **H1:** la media del puntaje de matemáticas difiere entre ambos grupos (μ_completed ≠ μ_none).

> 👴 **Explicado simple:** algunos estudiantes tomaron un cursito antes del examen, y otros no. Queremos saber: ¿de verdad ese curso ayuda a sacar mejor nota en matemáticas, o es pura casualidad? Es como preguntarte si un compañero que estudia con tutor antes de un examen le va mejor porque el tutor ayuda, o porque ya era bueno desde antes. Nuestra apuesta: **H0** dice "el curso no cambia nada"; **H1** dice "el curso sí cambia la nota". Vamos a usar los datos para ver cuál apuesta gana.

3. Identificación y clasificación de variables

- **Variable dependiente:** `math score` — numérica continua (0-100).
- **Variable independiente:** `test preparation course` — categórica con 2 grupos (`completed` , `none`).

In [4]:
```
# Nos quedamos solo con las dos columnas que nos importan para esta pregunta.
variables = df[["math score", "test preparation course"]]

# .dtypes nos dice de qué tipo es cada columna: si es un número (int64) o un texto (str).
print(variables.dtypes)

# .unique() nos muestra, sin repetir, cuáles son las "opciones" que puede tener esa columna de texto.
print("\nCategorías de la variable independiente:", df["test preparation course"].unique())
```

```
math score                 int64
test preparation course      str
dtype: object

Categorías de la variable independiente: <StringArray>
['none', 'completed']
Length: 2, dtype: str
```

> 👴 **Explicado simple:** de las 8 fichas de información, usamos dos: la **nota de matemáticas** (lo que queremos explicar, como la altura de una planta) y **si tomó o no el curso** (lo que podría estar influyendo, como si regaste o no regaste esa planta).

4. Descripción de la muestra y tamaño de los grupos

In [5]:
```python
# len(df) cuenta cuántas filas (estudiantes) hay en total en la tabla.
n_total = len(df)

# .value_counts() cuenta cuántas veces aparece cada opción ("none" o "completed") en esa columna.
conteo_grupos = df["test preparation course"].value_counts()

print(f"Tamaño total de la muestra (n): {n_total}")
print("\nTamaño por grupo:")
print(conteo_grupos)

# Aquí sacamos el porcentaje que representa cada grupo sobre el total (regla de tres simple).
print(f"\nProporción: {(conteo_grupos / n_total * 100).round(1)}")
```

```
Tamaño total de la muestra (n): 1000

Tamaño por grupo:
test preparation course
none         642
completed    358
Name: count, dtype: int64

Proporción: test preparation course
none         64.2
completed    35.8
Name: count, dtype: float64
```

No se aplicó ningún criterio adicional de inclusión/exclusión: se utilizó la totalidad de los 1000 registros del *dataset*, dado que no había valores nulos ni duplicados en las variables analizadas. La muestra queda compuesta por **642 estudiantes** que no tomaron el curso (`none`) y **358 estudiantes** que sí lo completaron (`completed`).

> 👴 **Explicado simple:** de los 1000 estudiantes, **642 no tomaron** el curso y **358 sí lo tomaron**. No quitamos ni agregamos ninguna ficha: usamos las 1000 tal como venían.

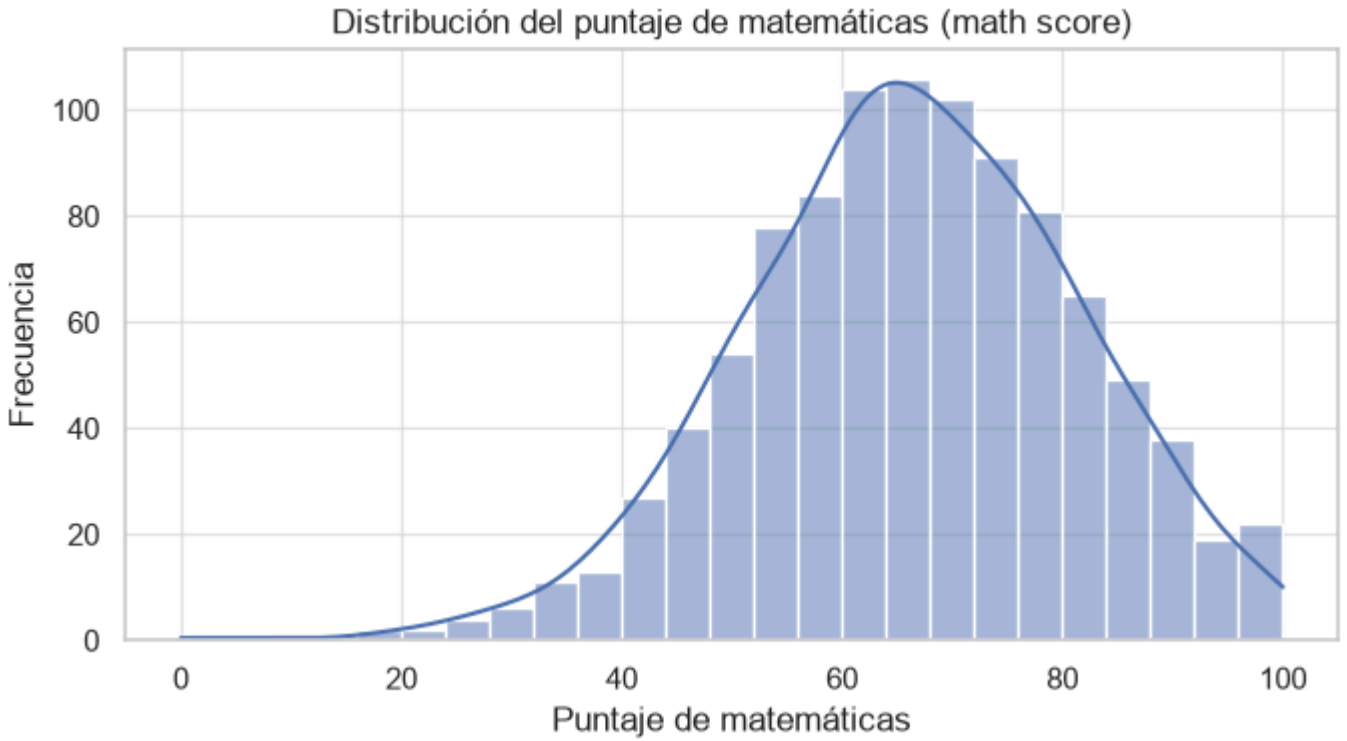
5. Estadística descriptiva y visualización

In [6]:
```
# .groupby() separa la tabla en dos "montoncitos": uno con los que tomaron el curso y otro con los que no.
# Luego, para cada montoncito, .describe() calcula el promedio, cuánto varían las notas, el mínimo y el máximo.
# Es como pedirle a dos grupos por separado su "resumen de notas de matemáticas".
desc_por_grupo = df.groupby("test preparation course")["math score"].describe()
desc_por_grupo
```

Out[6]:

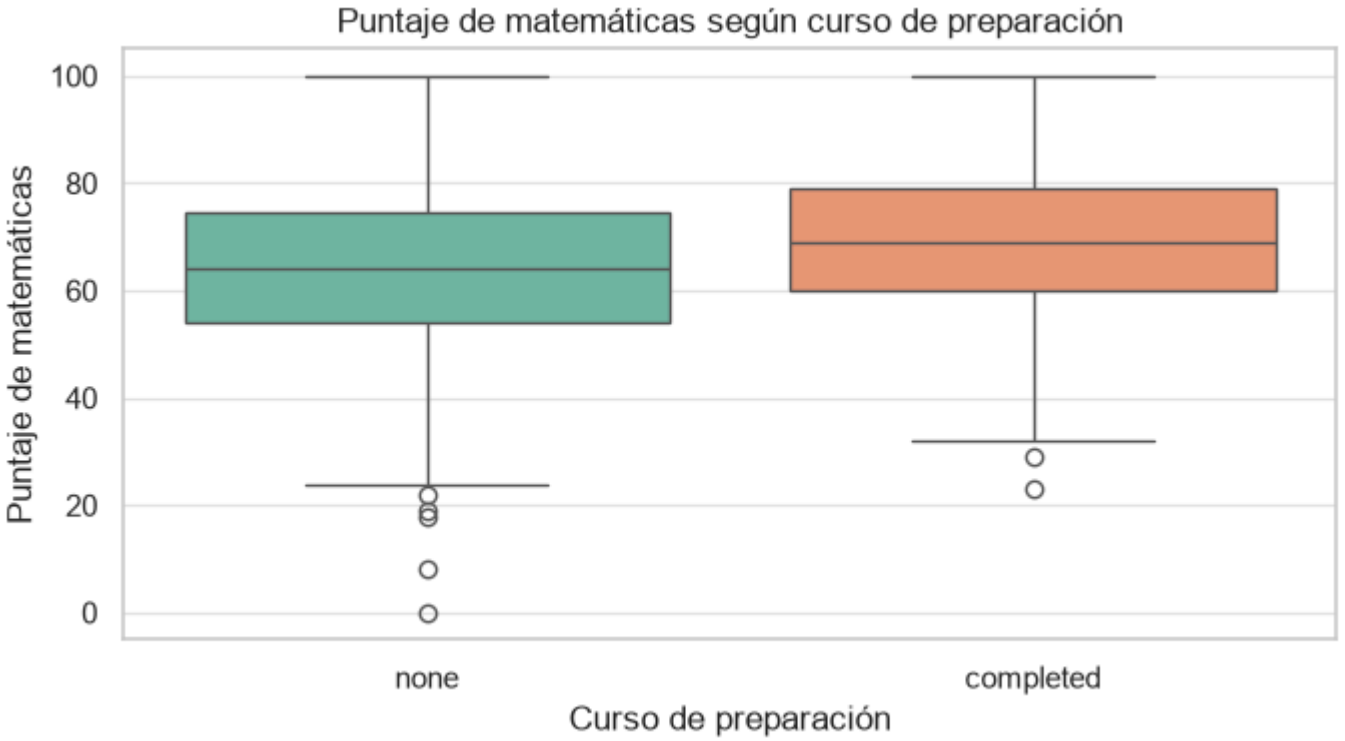
test preparation course	count	mean	std	min	25%	50%	75%	max
completed	358.0	69.695531	14.444699	23.0	60.0	69.0	79.00	100.0
none	642.0	64.077882	15.192376	0.0	54.0	64.0	74.75	100.0

In [7]:
```
# Un histograma es como una torre de bloques: agrupa las notas parecidas y apila un bloque por cada
# estudiante que sacó una nota en ese rango. Mientras más alta la torre, más estudiantes sacaron esa nota.
fig, ax = plt.subplots(figsize=(7, 4))          # preparamos una hoja en blanco para dibujar
sns.histplot(data=df, x="math score", kde=True, color="#4C72B0", ax=ax)  # dibujamos la torre de bloques
ax.set_title("Distribución del puntaje de matemáticas (math score)")     # título del dibujo
ax.set_xlabel("Puntaje de matemáticas")          # nombre del eje horizontal
ax.set_ylabel("Frecuencia")                      # nombre del eje vertical (cuántos estudiantes)
plt.tight_layout()
plt.show()                                        # mostramos el dibujo en pantalla
```



In [8]:
```
# Un boxplot (caja y bigotes) dibuja, para cada grupo, una cajita que muestra dónde está "la mayoría" de
# las notas y una línea en el medio que marca el valor típico (la mediana). Sirve para comparar dos grupos
# de un vistazo: si una caja está más arriba que la otra, ese grupo tiende a sacar mejores notas.
fig, ax = plt.subplots(figsize=(7, 4))
sns.boxplot(data=df, x="test preparation course", y="math score",
            order=["none", "completed"], hue="test preparation course",
            palette="Set2", legend=False, ax=ax)
```

```
ax.set_title("Puntaje de matemáticas según curso de preparación")
ax.set_xlabel("Curso de preparación")
ax.set_ylabel("Puntaje de matemáticas")
plt.tight_layout()
plt.show()
```



In [9]:
```
# Un scatter (gráfico de puntos) dibuja un punto por cada estudiante: dónde queda ese punto depende de sus
# dos notas (lectura y escritura). Si los puntos forman una "línea" imaginaria de abajo-izquierda a
# arriba-derecha, quiere decir que cuando una nota sube, la otra también tiende a subir.
fig, ax = plt.subplots(figsize=(6, 5))
sns.scatterplot(data=df, x="reading score", y="writing score", alpha=0.5, ax=ax)
ax.set_title("Relación entre puntaje de lectura y escritura (análisis complementario)")
plt.tight_layout()
plt.show()

# .corr() calcula un número entre -1 y 1 que resume qué tan "de la mano" van las dos notas.
# Mientras más cerca de 1, más fuerte es esa relación.
```

```
corr = df["reading score"].corr(df["writing score"])
print(f"Coeficiente de correlación de Pearson (reading vs writing): {corr:.3f}")
```




```
Coeficiente de correlación de Pearson (reading vs writing): 0.955
```

Lectura de las visualizaciones: el histograma muestra una distribución del puntaje de matemáticas razonablemente simétrica, cercana a una campana, aunque con cierta variabilidad. El *boxplot* evidencia que el grupo `completed` tiene una mediana y un promedio superiores al grupo `none`, con una dispersión similar entre ambos grupos. El *scatter* complementario confirma una fuerte correlación positiva entre los puntajes de lectura y escritura, lo que sugiere que ambas habilidades están muy relacionadas entre sí (no forma parte de la pregunta inferencial principal, pero aporta contexto adicional sobre el *dataset*).

> 👴 **Explicado simple:** primero resumimos lo que ya vemos, como contar tus dulces y sacar el promedio: el grupo que **no** tomó el curso promedia **64.08** puntos; el que **sí** lo tomó promedia **69.70** puntos (casi 5.6 puntos de diferencia). Los dibujos ayudan a "ver" esto: el histograma muestra una campanita, el *boxplot* muestra la caja de "sí tomó el curso" un poco más arriba, y el gráfico extra muestra que leer bien y escribir bien van muy de la mano. Pero ojo: ver una diferencia todavía no basta — hay que comprobar si es real o pura casualidad, porque 642 y 358 estudiantes son solo una *muestra*, no todos los estudiantes del mundo.

6. Validación de supuestos (inferencial)

6.1 Normalidad (Shapiro-Wilk)

In [10]:
```
# Separamos las notas de matemáticas en dos listas: las de quien NO tomó el curso y las de quien SÍ lo tomó.
grupo_none = df.loc[df["test preparation course"] == "none", "math score"]
grupo_completed = df.loc[df["test preparation course"] == "completed", "math score"]

# stats.shapiro() es la prueba que revisa si un grupo de notas forma una "campanita" (normalidad).
# Nos devuelve dos números: un "statistic" (no lo usamos directamente) y el p-value que sí nos interesa.
shapiro_none = stats.shapiro(grupo_none)
shapiro_completed = stats.shapiro(grupo_completed)

print(f"Shapiro-Wilk grupo 'none'      -> statistic={shapiro_none.statistic:.4f}, p-value={shapiro_none.pvalue:.4g}")
print(f"Shapiro-Wilk grupo 'completed' -> statistic={shapiro_completed.statistic:.4f}, p-value=
{shapiro_completed.pvalue:.4g}")
```

```
Shapiro-Wilk grupo 'none'      -> statistic=0.9921, p-value=0.001754
Shapiro-Wilk grupo 'completed' -> statistic=0.9937, p-value=0.1393
```

Interpretación: con $\alpha = 0.05$, el grupo `none` obtiene *p-value* < 0.05, por lo que **se rechaza el supuesto de normalidad** en ese grupo. El grupo `completed` obtiene *p-value* ≥ 0.05, por lo que **no se rechaza la normalidad** en ese grupo (con cautela, dado el tamaño muestral). Al no cumplirse el supuesto de normalidad de manera conjunta en ambos grupos, en sentido estricto correspondería usar una prueba **no paramétrica**.

6.2 Homogeneidad de varianzas (Levene)

In [11]:
```
# stats.levene() es la prueba que revisa si los dos grupos tienen una "variedad" parecida (homogeneidad):
# es decir, si en ambos grupos las notas están igual de "esparcidas" alrededor del promedio.
levene_test = stats.levene(grupo_none, grupo_completed)
print(f"Levene -> statistic={levene_test.statistic:.4f}, p-value={levene_test.pvalue:.4g}")
```

```
Levene -> statistic=0.5330, p-value=0.4655
```

Interpretación: el *p-value* de la prueba de Levene es ≥ 0.05, por lo que **no se rechaza el supuesto de homogeneidad de varianzas**: ambos grupos presentan una variabilidad estadísticamente comparable en el puntaje de matemáticas.

> 👴 **Explicado simple:** antes de usar una herramienta, hay que probar el agua con el dedo antes de meterse a la piscina. Revisamos si cada grupo forma una "campanita" perfecta (normalidad): en el grupo que **no** tomó el curso, la campanita **no** salió perfecta; en el grupo que **sí** lo tomó, sí salió razonablemente bien. También revisamos si los dos grupos tienen una variedad parecida (homogeneidad): **sí** la tienen. Como uno de los grupos no formó campanita perfecta, no podemos confiar del todo en la herramienta "normal" — toca usar una un poco más cuidadosa.

7. Selección y justificación del análisis inferencial

- **Tipo de variables:** una variable dependiente numérica continua (`math score`) y una variable independiente categórica de 2 grupos independientes (`test preparation course`) → escenario de **comparación de dos medias/dos muestras independientes**.
- **Normalidad:** no se cumple de forma conjunta en ambos grupos (el grupo `none` la incumple).
- **Homogeneidad de varianzas:** sí se cumple (Levene no significativo).
- **Tamaño de muestra:** grande en ambos grupos (n=642 y n=358), lo que por el Teorema del Límite Central hace que la prueba t sea razonablemente robusta ante desviaciones moderadas de normalidad.

Decisión: dado que el supuesto de normalidad no se cumple estrictamente en uno de los grupos, se aplica como prueba principal la alternativa **no paramétrica de Mann-Whitney U** (más conservadora y válida sin supuesto de normalidad). Como complemento, y aprovechando que las varianzas son homogéneas y el tamaño muestral es grande, también se reporta la **prueba t de Student para muestras independientes** a modo de verificación de robustez.

In [12]:
```python
# Prueba principal: Mann-Whitney U (no paramétrica) — la herramienta que "no necesita" que los datos
# formen una campanita perfecta. Compara los dos grupos y nos da un p-value.
mannwhitney = stats.mannwhitneyu(grupo_none, grupo_completed, alternative="two-sided")
print(f"Mann-Whitney U -> statistic={mannwhitney.statistic:.1f}, p-value={mannwhitney.pvalue:.4g}")

# Prueba complementaria: t de Student (varianzas homogéneas) — la herramienta "clásica" que usamos de
# chequeo extra, ya que con 1000 datos es razonablemente confiable aunque un grupo no sea perfectamente normal.
ttest = stats.ttest_ind(grupo_none, grupo_completed, equal_var=True)
print(f"t de Student   -> statistic={ttest.statistic:.4f}, p-value={ttest.pvalue:.4g}, df={ttest.df:.0f}")
```

```
Mann-Whitney U -> statistic=91424.0, p-value=8.015e-08
t de Student   -> statistic=-5.7046, p-value=1.536e-08, df=998
```

> 👴 **Explicado simple:** como un grupo no se comportó como campanita, usamos una herramienta que no necesita esa forma perfecta: **Mann-Whitney U**, como una regla que funciona bien aunque el papel esté un poco arrugado. Como tenemos MUCHOS datos (1000 en total), también probamos la herramienta clásica (**t de Student**) de chequeo extra.

8. Resultados e interpretación

Prueba	Estadístico	*p-value*	Decisión (α = 0.05)
Mann-Whitney U	91424.0	8.02e-08	Se rechaza H0
t de Student	-5.7046 (df=998)	1.54e-08	Se rechaza H0

Ambas pruebas —la principal (no paramétrica) y la complementaria (paramétrica)— coinciden en el mismo resultado: el *p-value* es muy inferior a α = 0.05, por lo que **se rechaza la hipótesis nula (H0)**.

Interpretación en función del problema planteado: existe evidencia estadísticamente significativa de que el puntaje de matemáticas difiere entre los estudiantes que completaron el curso de preparación y los que no. El grupo `completed` obtiene, en promedio, un puntaje de matemáticas de **69.70** puntos, frente a **64.08** puntos del grupo `none` (diferencia de ≈5.6 puntos). Es decir, los datos son consistentes con la idea de que tomar el curso de preparación está asociado con un mejor desempeño en matemáticas. Cabe aclarar que, al tratarse de datos observacionales (no de un experimento aleatorizado), esta asociación no permite afirmar causalidad de forma concluyente.

> 👴 **Explicado simple:** las dos herramientas dieron un resultado casi cero (0.00000008 y 0.0000000154): muchísimo más pequeño que 0.05, nuestra regla del juego. Sería casi imposible ver una diferencia tan grande solo por casualidad, así que le creemos a H1: **el curso de preparación SÍ está relacionado con una mejor nota en matemáticas** (69.70 vs. 64.08 puntos). Cuidado: como no hicimos un experimento de verdad (no elegimos al azar quién tomaba el curso), solo podemos decir que están **relacionados**, no que el curso *cause* la mejora al 100% — a lo mejor los que se apuntan al curso ya eran más estudiosos desde antes.

9. Conclusiones

- **Sobre los supuestos estadísticos:** validar normalidad y homogeneidad de varianzas antes de elegir una prueba inferencial es indispensable: en este caso, la normalidad no se cumplió estrictamente en uno de los grupos, lo que llevó a preferir una prueba no paramétrica como principal. Aun así, gracias al tamaño de muestra grande, la prueba paramétrica complementaria llegó a la misma conclusión, lo que reforzó la confianza en el resultado.
- **Limitaciones del *dataset*:** los datos son observacionales y transversales (no experimentales), por lo que no permiten establecer causalidad entre tomar el curso de preparación y obtener mejores puntajes; podrían existir variables de confusión no registradas (por ejemplo, motivación del estudiante, apoyo familiar o acceso a recursos educativos) que expliquen parte de la diferencia observada. Además, el tamaño de los grupos es desbalanceado (642 vs. 358).
- **Análisis adicionales o mejoras futuras:** con más datos o variables adicionales se podría: (1) aplicar un análisis de regresión múltiple o ANOVA de dos factores incluyendo variables como género o nivel educativo de los padres; (2) evaluar el efecto del curso de preparación sobre los tres puntajes (matemáticas, lectura y escritura) de forma conjunta; y (3) explorar diseños cuasi-experimentales o de control de variables confusoras para acercarse a una estimación causal más robusta.

> 👴 **Explicado simple:** aprendimos que antes de usar una herramienta hay que revisar si los datos se comportan como se espera. También aprendimos que estos datos no nos dejan estar 100% seguros de que el curso *causa* la mejora (no sabemos por qué cada quien decidió tomarlo o no), y que hay más estudiantes en un grupo (642) que en el otro (358). Con más información se podría revisar si el curso también ayuda en lectura y escritura, o incluso hacer un experimento de verdad para hablar de causa con más seguridad.

> **Resumen de una línea por paso** — ver la tabla completa en `Desarrollo_Actividad_Explicado_Nino.md`.

Referencias

- Bedre, R. (2021). *ANOVA using Python (with examples)* [Entrada en blog]. Data science blog. https://www.reneshbedre.com/blog/anova.html
- Boschetti, A., & Massaron, L. (2018). *Python data science essentials: A practitioner's guide covering essential data science principles, tools, and techniques*. Packt Publishing Ltd.
- Cady, F. (2020). *Data science: The executive summary - A technical book for non-technical professionals*. John Wiley & Sons, Incorporated.
- Fuentes, A. (2018). *Become a python data analyst: Perform exploratory data analysis and gain insight into scientific computing using python*. Packt Publishing, Limited.
- Guest Blog. (2020, 8 de junio). *Introduction to ANOVA for statistics and data science (with COVID-19 case study using Python)* [Entrada en blog]. Analytics Vidhya. https://www.analyticsvidhya.com/blog/2020/06/introduction-anova-statistics-data-science-covid-Python/
- Kaggle / spscientist. (s.f.). *Students performance in exams* [Conjunto de datos]. https://www.kaggle.com/datasets/spscientist/students-performance-in-exams